

Does Machine Learning Beat a Linear Factor Combination? A Pre-Registered, Cost-Aware Test

Rob Basnet

Draft: 2026-09-23; corrected 2026-10-07

Abstract

We pre-registered a single question: does a machine-learning model combining momentum, value, and quality beat a plain equal-weight linear combination of the same three factors, out of sample, in US large-cap stocks, after trading costs? On a point-in-time S&P 500 universe (2010–2024), tested walk-forward with purge/embargo gaps across a pre-committed grid of 9 configurations (8 machine-learning models plus the linear baseline), it does not: every machine-learning configuration loses to a baseline that itself loses money (out-of-sample Sharpe -0.590 to -0.361 , against -0.174 for the baseline). **This version (corrected 2026-10-07) corrects the version of this paper published on 2026-09-23**, which reported the best configuration at Sharpe 0.24 against a baseline at 0.07 . Both figures were artifacts of a look-ahead leak: the value factor divided as-filed EPS by split- and dividend-adjusted prices, inflating each stock’s earnings yield by its future split factor. The published edge, which this paper had already traced to a single 12-month window, now has a documented cause: correcting the feature, with nothing else changed, takes that window’s summed net monthly return from $+0.280$ to $+0.001$. The re-run also showed the published 0.24 was fragile in two further ways — re-downloading the same data alone moves it to 0.096 , and an unpinned scikit-learn version moves it to 0.147 . The pre-registered null survives the correction and is strengthened by it. A power analysis shows this design could not have detected a genuine Sharpe 0.3 – 0.5 edge in any case. **A data limitation also belongs up front: 30.7% of point-in-time S&P 500 members (213 of 694) are excluded for lack of price history, disproportionately delisted, acquired, and renamed names; for a dollar-neutral long/short design the direction of the resulting bias is ambiguous, and we do not attempt to sign it.**

Corrections

Correction 1 (2026-10-07): a look-ahead leak in the value factor

This section corrects the version of the paper published on 2026-09-23. The published figures are kept throughout the paper beside the corrected ones; none has been replaced. Every corrected number below is from `results/correction/` (`FACTS.md` gives each number with the table it comes from).

The defect. The value factor (earnings yield) divided TTM diluted EPS *as filed* by a price adjusted for every split and dividend up to the date the data was downloaded. After a split, earlier EPS is on the old share basis while the price is on the new one, so each stock’s earnings yield was

inflated by its *future* split factor, and more mildly by its future dividends. That is a look-ahead leak: a value score dated `t` used information from after `t`. Momentum and quality are unaffected (they are price ratios and company totals). Value feeds both arms — the linear baseline and every ML configuration consume `val_z` — so both were affected.

How it was found and corrected. The defect was in the backtesting engine this study vendors (factor-backtester at `31b92b4`). It surfaced while extending that engine with a size factor, where the same as-filed-versus-adjusted mismatch made market capitalisation (as-filed share counts times price) incomputable; the existing value factor was found to carry it already. It was fixed there, in `d68487a`: EPS is restated across later splits and divided by a split-adjusted but not dividend-adjusted price. The same mixing of share bases produced two artifacts the fix also removes — AAPL’s TTM EPS “falling” 61% after its 2020 split, and a derived NVDA Q4 FY2022 of $-\$1.09$ (actual $+\$1.18$). We then re-ran this study in the environment the published results were produced with (Python 3.12.9, scikit-learn 1.5.1, pandas 2.3.2, numpy 2.0.1, scipy 1.15.2; now pinned in `backtester/requirements.txt`). Before changing anything, the re-run reproduced all nine published return series exactly (maximum absolute difference 0 in every month) and the published factor IC table. It then ran the same pipeline on the same pre-registered grid, refitting every model in every fold, in two further modes: the published formula on re-downloaded data (isolating data drift) and the corrected formula. `n_trials` stays 9: the same nine configurations were recomputed, not a new search. The feature swap did only what it should: the mean monthly rank correlation between the leaky and corrected value scores is 0.955 for the 446 names that never split in 2008–2024 (where only the dividend part of the fix applies) and 0.680 for the 201 that split at least once.

Two problems in the measurement itself were caught and fixed. The first runs were made online and in parallel; Yahoo throttling caused the vendored loader to skiplist five stocks with cached prices mid-run (MCHP, KDP, HPQ, COST, HD), because it marks a ticker permanently unavailable whenever a request fails — a bug fixed in factor-backtester at `befc5c5`, after the vendored snapshot. Those numbers were discarded; every number here comes from an offline harness whose `published` mode reproduces exactly. Separately, the first committed corrected baseline lost one ticker to a concurrent cache rewrite (Sharpe -0.1728 instead of -0.1739); a disagreeing independent run exposed it, and it was replaced with a sequential re-run that matches that run exactly. The harness now reads the cache without writing to it and stops on any read error.

Published and corrected results. Walk-forward out-of-sample, net of 8 bps costs:

	Published	Published formula, re-downloaded data	Corrected
Baseline Sharpe	0.066	0.052	−0.174
Baseline deflated Sharpe, n=1 / n=9	0.578 / 0.093	0.561 / 0.086	0.294 / 0.020
Best ML configuration	rf_depth10_leaf50	rf_depth10_leaf50	gbm_lro.03_depth3
Best ML Sharpe	0.241	0.096	−0.361
Best ML deflated Sharpe, n=9	0.202	0.108	0.005
Best ML max drawdown	−32.5%	−36.5%	−55.8%
ML Sharpe range, 8 configurations	−0.357 to 0.241	−0.353 to 0.096	−0.590 to −0.361
ML configurations beating the baseline	3 of 8	1 of 8	0 of 8

Per configuration (out-of-sample Sharpe):

Configuration	Published	Published formula, re-downloaded data	Corrected
Baseline (linear)	0.066	0.052	−0.174
gbm_lro.03_depth3	0.160	−0.001	−0.361
gbm_lro.03_depth5	−0.036	−0.072	−0.517
gbm_lro.10_depth3	0.066	−0.125	−0.421
gbm_lro.10_depth5	0.145	0.018	−0.590
rf_depth5_leaf50	−0.009	−0.064	−0.404
rf_depth5_leaf200	−0.357	−0.353	−0.423
rf_depth10_leaf50	0.241	0.096	−0.457
rf_depth10_leaf200	−0.220	−0.257	−0.471

The published “3 of 8” rests on 0.0004: gbm_lro.10_depth3 (0.06564) is below the baseline (0.06606), but Section 5.1’s table prints both as 0.066, so a reader counting from that table gets four.

What the correction changes. The conclusion keeps its direction and is stronger. The pre-registered null survives; every ML configuration now loses to the baseline outright, before any multiple-testing correction; and the baseline it loses to is itself negative. The published best configuration’s apparent edge now has a documented cause. In the full-sample factor IC (Section 6), value was the only factor with $|t| > 1$ (mean 0.0111, $t = 1.26$); corrected, its mean IC is −0.0051 ($t = -0.48$), and no factor clears $|t| = 1$.

Fold 7. Section 5.2 found that rf_depth10_leaf50’s entire published edge came from fold 7 (2022-09 to 2023-09). Its summed net monthly return in that fold is +0.280 published, +0.244

with the published formula on re-downloaded data, and +0.001 corrected (baseline: −0.067, −0.078, −0.123). A gross attribution of the published fold-7 return by later forward splits — names that split between 2023-09-01 and 2024-12-31, whose value scores the leak inflated during the fold — finds that the long positions in those names contributed +0.046 of the +0.289 gross total (8 names, 7.2% of long-leg weight) and the short positions in them −0.006 (7 names, 2.7% of short-leg weight). Holding later-splitters in the test window is therefore not the mechanism. The edge disappears once the feature is corrected with nothing else changed (corrected gross total +0.010), which leaves contamination of the training data — the leaky value-to-return relationship in the history each model was fitted on — as the explanation. That last step is an inference from these two measurements, not a direct measurement of what the models learned.

Three separate fragilities. The re-run exposed three independent ways the published 0.241 depended on things it should not have. For `rf_depth10_leaf50`, with everything else unchanged:

Change	Out-of-sample Sharpe
As published	0.241
Data re-downloaded, leaky formula unchanged (data vintage)	0.096
scikit-learn 1.9 instead of 1.5.1, same code and data	0.147
Leak corrected	−0.457

Only the leak is a bug. Data vintage and library version are robustness failures: over the same data-vintage change the baseline moved only from 0.066 to 0.052, while the published best configuration lost 60% of its Sharpe. The library version is now pinned; the data-vintage sensitivity is reported here, not removed.

Exploratory, not a finding: did the ML configurations lose more than the baseline?

This comparison was formed after seeing the results and rests on $n = 8$ configurations; it is an observation, not a finding. From published to corrected, the baseline’s Sharpe changed by −0.240. The mean ML change was −0.454, 1.89 times the baseline’s, and 7 of 8 configurations changed by more than the baseline. `rf_depth10_leaf50` changed by −0.697 (2.91 times); it was selected for looking best, so it is the configuration most expected to have exploited the artifact. The two most regularised forests changed least (`rf_depth5_leaf200` −0.066, `rf_depth10_leaf200` −0.251), which is consistent with flexibility mattering but is not a test of it. Paired tests find no significant difference in mean monthly return impact relative to the baseline for any configuration (t from −1.56 to +0.79; $|t| < 2$ for all eight); that test is on mean monthly return impact, not on the Sharpe swings themselves.

Ticker identity, narrowed. Section 2 and Limitation 3 originally said we “checked every structural pattern” under which ticker matching could pair the wrong entity’s data with a valid membership window, and found none. The check was narrower than that. It examined 20 names — the 3 with non-contiguous membership runs (AMD, DXC, GAS) and 17 previously flagged tickers — on the premise that a non-contiguous run “is the only structural pattern under which one ticker symbol could span two different companies.” COL contradicts that premise. Rockwell

Collins (COL) was an S&P 500 member from 2010-01-01 to 2018-11-26, in a single contiguous run, yet the study's cached COL series runs from 2012-08 to 2020-11, and over the 75 months it overlaps Rockwell Collins's own price series the monthly returns correlate at 0.069. The cache holds a different security under the same symbol for COL's membership from 2012-08 on, and nothing for 2010-01 to 2012-07; which security it is has not been established. COL's effect on the results has not been isolated: it is one of about 480 names; in every mode its returns and momentum come from the contaminated cache, and in the two re-downloaded modes its value input comes from Rockwell Collins. Ticker matching cannot give the guarantee the original wording claimed; only a permanent security identifier, such as CRSP PERMNO, would.

Which correction is which. Section 5.2 also contains an earlier correction, made before publication: a draft read `rf_depth10_leaf50`'s 7-of-9 fold win rate as evidence of a broad-based edge, and the leave-one-fold-out analysis overturned that reading. That in-draft correction stands as written. It is about how to read the published results, and it was right about them — the edge was one fold. The leak correction sits on top of it and explains what it could not: why that fold. Section 5.2 now labels the two.

What was and wasn't re-run. The correction re-ran the return series, per-configuration Sharpe, the headline deflated Sharpe and drawdown figures above, fold-level returns, the leave-one-fold-out table (5.2), and the factor IC. Permutation importance (5.3), the power analysis (5.4), the gross-of-cost and cost-sensitivity checks, and the remaining columns of the Section 5.1 table describe the published run only, and are marked where they appear.

One further correction. The Appendix described the repository as private during drafting, to be made public later. It was public throughout.

1. Introduction

Combining multiple return-predicting signals into a single trading signal is a basic move in quantitative equity investing, and it raises an old question in a new form: is a hand-picked linear combination — an equal-weight average, in the simplest case — leaving money on the table that a flexible, non-linear model could capture? Machine learning has been applied to this question at scale in recent years: Gu, Kelly, and Xiu (2020), working with hundreds of firm characteristics across the full cross-section of US equities, report annualized out-of-sample Sharpe ratios of 1.35 (value-weighted) to 2.45 (equal-weighted) for a neural-network long-short decile strategy, against 0.61 to 0.83 for an OLS benchmark on the same features — a large gain, though reported gross of transaction costs on a paper portfolio, not as a tradeable net return. This paper asks the same question in a much smaller, much more reproducible, and explicitly cost-aware setting: three well-known factors — momentum, value, and quality — combined by an off-the-shelf gradient boosting or random forest model, tested walk-forward, after realistic trading costs, in US large-cap stocks.

We pre-registered the question, the hypothesis (stated as a null, on purpose), the exact model grid, and the win condition before training any model (`PREREGISTRATION.md` , committed 2026-07-07, three weeks before any machine-learning result existed — see the Appendix for the exact commit

history). The short answer, as published: no configuration in the pre-registered grid beats the linear baseline once the honest cost of searching over 9 configurations is priced in, and two independent robustness checks — leave-one-fold-out and per-fold permutation importance — both point to the same window and the same factor. The short answer, corrected (Corrections): no configuration beats the baseline even before that cost is priced in, the baseline itself is negative, and the window and factor those checks pointed to trace to a look-ahead leak in the value factor.

2. Data

Universe. Point-in-time S&P 500 membership, 2010-01-01 to 2024-12-31, built from the community-maintained `fja05680/sp500` historical components-and-changes dataset. Membership is a boolean (`date × ticker`) matrix that includes names that were later delisted, acquired, or renamed — 694 distinct tickers appear as point-in-time members over the window, and a ticker’s presence in the matrix on a given date requires only that it was logged as a member as of that date, never knowledge of a later removal.

Prices. Daily adjusted close from Yahoo Finance (`yfinance`, keyless), with Tiingo’s Daily Prices API available as a documented fallback for names Yahoo has dropped. Prices are resampled to month-end and pivoted to a (`date × ticker`) panel; forward returns are `pct_change().shift(-1)`, so the return paired with a rebalance date is always the return earned *after* that date, never the return that produced the signal.

Fundamentals. SEC EDGAR XBRL company facts (keyless), giving TTM diluted EPS, StockholdersEquity, and NetIncomeLoss, from which we derive earnings yield (value) and ROE (quality). Every fundamental value is lagged to its SEC filing date plus a 90-day buffer before it can affect any factor score on or after that date — this, not merely using the filing date itself, is what prevents look-ahead from restated or late-arriving fundamentals.

Coverage gap: 30.7% of the universe is excluded from the panel actually traded on, for missing price history. Of the 694 point-in-time universe members, 213 (30.7%) have zero rows anywhere in the ML panel or the linear baseline’s composite, because Yahoo Finance has no price history for them and the Tiingo fallback did not resolve the gap (below). 196 of the 213 are recorded in a persistent skiplist (`unavailable_prices.json`); the remaining 17 (including `DOW`, `DELL`, `CEG`, `EMC`) were separately checked for a worse failure mode — a base ticker symbol carrying price data from a *different* company than the point-in-time member (e.g. new “Dow Inc.”, relisted 2019, attributed to old Dow Chemical’s 2010–2017 membership window) — and confirmed clean: every one of the 17 has price coverage that falls entirely *outside* its old membership window, so for the excluded names this is a missing-data problem, not a wrong-data problem. (An *included* name, `COL`, does carry another security’s prices; see Ticker identity below.) **The direction of the bias this gap introduces is ambiguous for a dollar-neutral long/short design, and we do not attempt to sign it.** Names that deteriorated into bankruptcy would mostly have sat in the short leg, so dropping them removes short-leg gains a real investor would have earned; names acquired at a premium would have produced a sharp adverse move against a short position, so dropping them removes short-leg losses a real investor would have

taken. These pull in opposite directions, and nothing in the available data lets us net them out. (The intuition that excluding delisted names simply inflates returns is a long-only intuition — it does not transfer to a dollar-neutral book, where the same names could equally have sat on the short leg.)

We obtained a Tiingo API key and, before committing to a full 196-name re-fetch, ran a 5-name spot check on confirmed-delisted names spanning the gap’s era (`ANTM`, `APC`, `CEPH`, `BNI`, `BMC`). Zero of five returned any price data; Tiingo’s own ticker-metadata endpoint confirms it correctly identifies each company (e.g. “BMC Software Inc”, flagged `DELISTED`) but has never ingested price history for them (`startDate` / `endDate` both null) — a structural gap in this data tier’s historical depth for this era, not an authentication or code issue (a control fetch of a currently-listed name on the same key returned data immediately). We stopped there rather than spending the API budget on a fuller re-fetch expected to reproduce the same result. Full detail, including the exact reproduction script, is in `results/coverage_gap_note.md` and `results/ticker_identity_check.md`.

A separate, smaller gap exists in fundamentals coverage: 216 tickers have no usable SEC EDGAR data (missing CIK resolution or a non-standard EPS tag), which does not remove a ticker from the panel but leaves its `val_z` / `qual_z` NaN on a given date. Composite score coverage across the study window averages 95.4% of the active universe per rebalance date (minimum 25.4%, concentrated in the earliest dates before 12 months of price history accumulate); value-and-quality coverage jointly averages 83.8% (minimum 18.6%). Per-date detail is in `results/baseline/coverage_report.csv`.

Provenance of these numbers. The counts and coverage figures in this section (694, 213, 196, 17, 216, and the four coverage percentages) come from the published run’s data cache — Yahoo-only, with no `TIINGO_KEY` — and were not re-derived for the correction. The 196-name skiplist was built by the vendored loader, which marked a ticker permanently unavailable whenever a request failed, not only when every source confirmed it had no data (fixed in factor-backtester at `befc5c5`, after the vendored snapshot); whether any of the 196 were skiplisted by a transient failure rather than a genuine absence is not established here. The correction’s value input comes from two panels built on re-downloaded data (published formula and corrected), with 76,950 and 76,624 non-NaN cells respectively; those counts are not directly comparable to the published run’s coverage percentages.

Ticker identity. All matching in this study is on ticker symbol, not a permanent identifier (CIK for fundamentals, or a security-level identifier like CRSP PERMNO for prices). We checked two patterns under which this could silently pair the wrong entity’s data with a valid membership window — tickers with non-contiguous membership runs (`AMD`, `DXC`, `GAS`), and 17 previously flagged tickers whose price fetch succeeded but excluded their old window — and found no contamination among those 20 names (`results/ticker_identity_check.md`). *Corrected 2026-10-07:* the published version of this paragraph said we had checked “every structural pattern.” That overstated the check, which rested on the premise that a non-contiguous membership run is the only pattern under which one symbol can span two companies. COL is a

counterexample: Rockwell Collins was a member in a single contiguous run (2010-01-01 to 2018-11-26), yet the cached COL series from 2012-08 on belongs to a different security (Corrections). Ticker matching cannot give the guarantee a permanent identifier would.

3. Factors

- **Momentum:** 12-month return, skipping the most recent month (12-1 momentum), the standard construction from Jegadeesh and Titman (1993).
- **Value:** earnings yield (TTM diluted EPS / price). *Published:* EPS as filed, divided by a price adjusted for every split and dividend up to the download date — the look-ahead leak described in Corrections. *Corrected:* EPS restated across later splits, divided by a split-adjusted but not dividend-adjusted price.
- **Quality:** return on equity (TTM net income / stockholders' equity).

Each factor is winsorized at the 1st/99th percentile and cross-sectionally standardized (z-scored) within each rebalance date's universe, following the Fama-French tradition of relative, not absolute, factor scores. The linear baseline is the equal-weight average of the three z-scores, re-normalized per name by whichever factors are actually present rather than propagating a NaN whenever one factor is missing.

4. Method

Portfolio construction. Monthly rebalance. Names are ranked by the composite score (linear) or model-predicted score (ML) into deciles; the strategy goes long the top decile and short the bottom decile, equal-weighted within each leg, dollar-neutral overall
(`portfolio.n_deciles = 10`, `long_short = true`).

Cost model. 8 basis points per unit of turnover, applied every rebalance (`costs.bps_per_trade = 8`), where turnover is $0.5 * \sum(|w_t - w_{(t-1)}|)$, with the pre-rebalance weight taken as cash (0) at the very first period of each walk-forward test block.

Walk-forward validation. Expanding-window folds: a 60-month initial training window, a 1-month embargo (no rebalance scored or traded in the gap), then a 12-month out-of-sample test block; each subsequent fold absorbs the prior fold's test block into training before applying a fresh embargo. This produced 10 folds (the last truncated to 1 OOS month by the sample's end) and 109 stitched out-of-sample periods total. Because nothing in the linear pipeline is *fit* to labels (no lookback, weight, or decile count is estimated from data), the embargo here exists primarily to keep factor lookback windows and lagged fundamentals from leaning on information still "in flight" at a fold boundary; for the ML pipeline, the same fold boundaries additionally ensure every model is fit on strictly prior data and evaluated on strictly subsequent data, with imputation (median-fill for a missing factor value) also fit on the training fold only. The 1-month embargo is sufficient for this specifically because each training row's label is the return from its rebalance date t to $t+1$ — a 1-month-ahead target — so no training label's realization window extends past the

single month the embargo excludes; a training row immediately adjacent to the test block, in other words, still resolves entirely before the test block opens.

Model grid (pre-registered, PREREGISTRATION.md).

Family	Hyperparameters	Configurations
Gradient boosting (HistGradientBoostingRegressor)	learning_rate $\in \{0.03, 0.10\} \times$ max_depth $\in \{3, 5\}$	4
Random forest (RandomForestRegressor)	max_depth $\in \{5, 10\} \times$ min_samples_leaf $\in \{50, 200\}$	4
Linear baseline	equal-weight composite	1

9 total configurations, fixed before any model was trained. This count is what the deflated Sharpe ratio’s multiple-testing correction ($n_trials=9$) is scored against.

Deflated Sharpe ratio. Following Bailey and López de Prado (2014), the deflated Sharpe ratio (DSR) answers: given that `n_trials` configurations were tried, what is the probability the *true* Sharpe ratio of the selected one is positive, after correcting for (a) the selection bias of picking the best of `n_trials` tries and (b) estimation noise from non-normal (skewed, fat-tailed) returns? We report DSR at $n_trials=9$ for every ML configuration and at $n_trials=1$ for the linear baseline, since the baseline is a fixed, pre-specified benchmark, not the result of a search — both conventions are reported side by side in Section 6 so neither obscures the other.

Win condition (pre-registered). Machine learning “wins” only if (1) its walk-forward, stitched, net-of-cost out-of-sample Sharpe ratio exceeds the baseline’s, *and* (2) that result survives the deflated Sharpe correction at $n_trials=9$. **The pre-registration committed to condition (2) in words (“survives the deflated Sharpe”) but never fixed a numerical threshold.** We adopt 0.95 as the threshold — the conventional cutoff for “likely genuine skill, not noise picked out of a search” — as a post-hoc but standard choice, made after seeing results, and disclose it as such rather than presenting it as pre-committed. The conclusion below is not sensitive to exactly where this line is drawn: the best configuration’s net deflated Sharpe is 0.2022, so “ML does not win” holds for any threshold ≥ 0.21 , comfortably below any conventional choice (0.5, 0.75, or 0.95 all give the same answer). Corrected, the question does not arise: no configuration meets condition (1), and the best configuration’s deflated Sharpe is 0.005.

5. Results

5.1 The win condition fails on the pre-registered terms

Config	OOS Sharpe	Corrected OOS Sharpe	Deflated Sharpe (n_trials)	Ann. return	Max drawdown	Hit rate	Turnover
Baseline (linear)	0.066	-0.174	0.578 (n=1)	-0.57%	-47.78%	55.96%	37.12%
gbm_lro.03_depth3	0.160	-0.361	0.146 (n=9)	1.23%	-38.05%	53.21%	71.59%
gbm_lro.03_depth5	-0.036	-0.517	0.052 (n=9)	-1.20%	-36.98%	46.79%	89.21%
gbm_lro.10_depth3	0.066	-0.421	0.093 (n=9)	-0.02%	-37.08%	54.13%	79.07%
gbm_lro.10_depth5	0.145	-0.590	0.137 (n=9)	1.01%	-28.34%	55.96%	97.27%
rf_depth5_leaf50	-0.009	-0.404	0.061 (n=9)	-1.11%	-44.77%	55.05%	82.48%
rf_depth5_leaf200	-0.357	-0.423	0.004 (n=9)	-5.26%	-53.55%	47.71%	81.50%
rf_depth10_leaf50 (best, published)	0.241	-0.457	0.202 (n=9)	2.14%	-32.52%	59.63%	100.57%
rf_depth10_leaf200	-0.220	-0.471	0.014 (n=9)	-2.79%	-39.97%	46.79%	100.38%

109 stitched out-of-sample periods, all configurations. Every column except “Corrected OOS Sharpe” describes the published run; the correction’s headline deflated Sharpe and drawdown figures are in Corrections, and the other columns were not re-tabulated. Two rows print as 0.066 published: the baseline is 0.06606 and gbm_lro.10_depth3 is 0.06564, so the latter does not beat the baseline — 3 of 8 configurations did in the published run, not four.

In the published run, `rf_depth10_leaf50` is the best of the 8 ML configurations by raw Sharpe (0.241 vs. the baseline’s 0.066) — condition (1) of the win condition is met — but its deflated Sharpe (0.202) falls far short of the 0.95 bar — condition (2) is not. Corrected, condition (1) fails for every configuration: the best, gbm_lro.03_depth3 at -0.361, is below the baseline’s -0.174. Both sides of the deflated-Sharpe convention are shown together (both arms scored at both `n_trials=1` and `n_trials=9`) in `results/comparison/README.md` so the asymmetric convention does not invite a misreading of which side is “better.” Gross-of-cost figures (`results/comparison/gross_vs_net.csv`, published run only) rule out an obvious alternative explanation: `rf_depth10_leaf50`’s gross deflated Sharpe (0.267) also falls far short of 0.95, so trading costs are not what turns an otherwise-winning result into a loss.

5.2 Leave-one-fold-out: the entire apparent edge is one 12-month window

Leak correction (2026-10-07). The text of this section — the analysis, and the in-draft correction at its end — describes the published run, whose value factor contained the look-ahead leak set out in Corrections. The table has been re-run for the correction and shows both: the first three numeric columns are the published run, and the last three are corrected (`results/correction/tables/leave_one_out.csv` , whose published rows reproduce the published table exactly). The published finding, that fold 7 carried the entire edge, stands as a description of that run, and the leak correction explains it: corrected, `rf_depth10_leaf50` 's summed net monthly return in fold 7 falls from +0.280 to +0.001 (baseline: -0.067 to -0.123). Corrected, there is no edge left to locate: `rf_depth10_leaf50` is below the baseline whichever fold is dropped.

Fold dropped	Baseline Sharpe	ML Sharpe	ML – baseline	Corrected baseline	Corrected ML	Corrected ML – baseline
0	-0.114	+0.269	+0.384	-0.325	-0.474	-0.149
1	+0.123	+0.290	+0.167	-0.099	-0.469	-0.370
2	-0.006	+0.208	+0.214	-0.249	-0.409	-0.160
3	+0.047	+0.234	+0.187	-0.206	-0.547	-0.341
4	-0.036	+0.162	+0.198	-0.279	-0.555	-0.277
5	+0.390	+0.646	+0.256	+0.137	-0.103	-0.240
6	-0.003	+0.279	+0.282	-0.241	-0.477	-0.237
7	+0.115	-0.024	-0.138	-0.107	-0.495	-0.387
8	+0.111	+0.192	+0.082	-0.155	-0.487	-0.332
9	+0.073	+0.224	+0.152	-0.168	-0.496	-0.328
None (full sample)	+0.066	+0.241	+0.175	-0.174	-0.457	-0.283

For each of the 10 walk-forward folds, we drop that fold's periods from the pooled out-of-sample series and recompute Sharpe on what remains, for both arms. **Dropping fold 7 (2022-09 to 2023-09) alone takes `rf_depth10_leaf50` from Sharpe 0.241 to -0.024** — the only fold whose removal flips the sign of ML's advantage, by a wide margin (the next-largest swing from dropping any other single fold is 0.09). In cumulative-return terms: ML's total advantage over the baseline across all 10 folds is +16.9 percentage points; fold 7 alone contributes +38.4pp to that (its own ML cumulative return of +31.3% against the baseline's -7.1% in the same window) — more than the entire net advantage. Every other fold, combined, nets to **-21.5pp**: ML *underperforms* the baseline once fold 7 is excluded, not merely loses its edge. The best-performing configuration's entire apparent edge over the linear baseline comes from a single 12-month window; excluding it, the baseline outperforms in aggregate. This is a statement about the sum across folds, not about each fold individually — fold-by-fold, ML's raw Sharpe still exceeds the baseline's in most of the

other 8 folds (full per-fold detail in `results/comparison/per_fold_subperiods.csv`), just by margins small enough that they cannot offset fold 7’s dominance of the total. Magnitude, not count, is what determines the aggregate — exactly the distinction the next paragraph’s self-correction is about.

In-draft correction (made before publication; distinct from the 2026-10-07 leak correction above). A methodological note on how we arrived at this reading. An earlier draft of this analysis observed that `rf_depth10_leaf50`’s Sharpe exceeded the baseline’s in 7 of 9 full walk-forward folds and read this win rate as evidence the pooled advantage was broad-based rather than concentrated in one window. That reading was incorrect: a simple win count treats every fold’s win or loss equally regardless of size. Four of those seven “wins” are near-rounding-error differences (e.g. Sharpe 1.090 vs. 1.082), and three of those four actually have *lower* cumulative return than the baseline — ML wins them on lower volatility, not better returns — while the two losses are large. The leave-one-fold-out analysis above is what overturned this reading, and we report the correction here rather than silently revise the earlier claim, consistent with this study’s pre-registration discipline (Appendix).

5.3 Permutation importance: no stable relationship with any factor

Published run only; not re-run for the correction (Corrections).

Per-fold permutation importance for `rf_depth10_leaf50`, scored by cross-sectional rank IC (Spearman correlation of predicted score vs. realized forward return, within each out-of-sample month, averaged over the fold) — not `feature_importances_`, which is computed on training data and is biased toward features with more split opportunities, and not R^2 on pooled returns, which this decile-ranking strategy does not actually depend on. Computed on the 9 full walk-forward folds (the 10th, truncated to a single out-of-sample month, is excluded — a single date’s rank IC is one Spearman correlation, not an average, and would be over-weighted in a mean/std computed across folds of unequal length).

Feature	Mean importance	Std across folds
val_z	0.0107	0.0203
qual_z	0.0030	0.0114
mom_z	0.0018	0.0130

The standard deviation of importance across folds exceeds the mean, for all three features. The feature with the highest importance changes fold to fold — `val_z` leads in 5 of 9 folds, momentum and quality in 2 each — with no consistent leader. In 3 of the 9 full folds, the model’s own (unpermuted) predictions were negatively rank-correlated with realized forward returns.

This converges with the leave-one-fold-out finding above, rather than merely sitting beside it: fold 7 — the single fold responsible for all of the model’s apparent edge — is also the fold where `val_z`’s importance peaks (0.059, roughly 3x any other fold) and

where the model’s baseline rank IC is the highest of any full fold. Two independent diagnostics — a portfolio-level robustness check and a feature-level stability check — locate the same fold as the source of the entire result. This reads as one window in which value happened to work, with no evidence it generalizes: `rf_depth10_leaf50` is not learning a stable relationship with momentum, value, or quality, and its headline 0.241 Sharpe is one regime’s bet, not a repeatable edge (corrected Sharpe: -0.457). Value “worked” in that window only with the leak in place: once the published value scores’ future-split inflation is removed, with nothing else changed, fold 7’s edge is gone. That the mechanism is contamination of the training history is an inference (Corrections), and whether the fold-7 peak in `val_z` importance itself came from the leak was not re-measured; it is consistent with it.

(Cost sensitivity, computed on the published run as a supplementary check and not re-run for the correction, found the pooled Sharpe advantage’s break-even cost at ~ 31.6 bps/trade, about 4x the study’s 8bps assumption. We do not treat this as supporting evidence: the advantage whose cost-robustness it measures is itself a fold-7 artifact per Section 5.2, so the crossover describes the durability of an effect that is not a genuine, broad-based edge to begin with. Full detail in `results/comparison/README.md`.)

5.4 Statistical power: could this design have detected a real edge?

Independent of whether ML actually beats the linear baseline, we ask whether this study’s design — 109 monthly out-of-sample observations, `n_trials=9` — could have detected a genuine edge of a plausible size in the first place. We invert the deflated Sharpe formula (root-finding the annualized Sharpe needed to reach a target DSR, given this sample’s actual size and each series’ own skew and kurtosis, rather than assuming a normal-returns approximation) to find the minimum detectable effect:

OOS months	Ann. Sharpe needed for $\text{DSR} \geq 0.50$	for $\text{DSR} \geq 0.95$
80	0.69	1.90
100	0.60	1.58
109 (this study)	0.57	1.48
118	0.55	1.39
130	0.52	1.30
150	0.48	1.17

Using `rf_depth10_leaf50`’s own empirical skew (-1.35) and kurtosis (9.43 , against 3 for a normal distribution; both from the published series, and this table was not re-run for the correction) — its out-of-sample returns are meaningfully negatively skewed and fat-tailed — this design needed an annualized out-of-sample Sharpe of roughly **1.5** to declare a win at $\text{DSR} \geq 0.95$. A genuine edge of annualized Sharpe 0.3 – 0.5 — well below Gu, Kelly, and Xiu’s (2020) headline gross figures (Section 1), but a plausible order of magnitude for a net-of-cost edge once turnover and trading frictions are priced in, as the cost-aware factor-investing literature generally finds —

would have been **undetectable** by this design at this sample size and this multiple-testing correction, independent of whether such an edge exists. This is a distinct claim from the null result itself: the study fails to reject the null, *and* the design could not have rejected it for any realistic effect size.

6. Discussion

Does ML beat the linear baseline? No, on the pre-registered terms, and the two robustness checks in Sections 5.2–5.3 point to why: the best configuration’s apparent edge is not a stable, broad-based improvement over the linear composite but the contribution of a single 12-month window in which one factor (value) appeared to work — and, per the correction, did so only with the look-ahead leak in place — with the underlying model otherwise showing no consistent relationship to any of the three factors across folds. Notably, this is *not* explained by the three factors being collinear and therefore offering a non-linear model little extra structure to combine — we checked, and they are nearly orthogonal in this sample (pairwise correlations of 0.02 between momentum and value, 0.03 between momentum and quality, and 0.13 between value and quality, on the full panel; because each factor is cross-sectionally z-scored per date, the pooled correlation and the average monthly cross-sectional correlation coincide here, so this figure needs no further adjustment).

The more direct explanation is that none of the three factors has a cross-sectional relationship with forward returns distinguishable from zero in this sample. `backtester/scripts/factor_ic.py` computes each factor’s monthly rank IC (Spearman correlation of the factor against next-month return, within each rebalance date’s cross-section, skipping months with fewer than 20 valid names) and its t-statistic (mean / (std / $\sqrt{n}$)) across all 167–179 such months in the full sample:

Factor	Months	Mean IC	Std IC	t-stat	Corrected mean IC (t)
mom_z	167	0.0036	0.209	0.22	unchanged
val_z	179	0.0111	0.118	1.26	−0.0051 (t = −0.48)
qual_z	179	0.0041	0.120	0.46	unchanged

The first five columns are the published run; momentum and quality do not depend on the corrected feature, so their ICs are unchanged. None of the three t-statistics clears even a loose significance bar, published or corrected. This supersedes an earlier draft’s use of pooled, non-cross-sectional Pearson correlations (−0.003, 0.012, −0.003) for the same claim — the wrong statistic for a cross-sectional decile strategy, since an un-demeaned `fwd_ret` carries a large common market component that pooling drags toward zero regardless of the cross-sectional signal actually present. The correction changes more than the numbers: two of the three pooled figures (momentum and quality) had the **wrong sign** relative to the correctly computed monthly rank IC. Both sets of statistics are full-sample descriptive properties of the factors — including periods the ML models were trained on, not just tested on — not an out-of-sample performance

claim; Section 5’s walk-forward results remain the paper’s evidence on that question. Two further observations from the corrected table converge with the walk-forward results above: momentum’s IC volatility (~ 0.21) is nearly double value’s and quality’s (~ 0.12) — its cross-sectional signal isn’t just weak on average, it is also far less stable month to month, the same fat-tailed, regime-dependent behavior that shows up as fold 5’s crash (Section 5.2) and that the Daniel and Moskowitz (2016) pattern already cited describes. And in the published run, value is the only factor with a t-statistic above 1 — consistent with `val_z` leading per-fold permutation importance in 5 of 9 folds and fold 7 (the single fold responsible for the entire ML edge) being a value-favoring regime (Section 5.3). Three independent views of the published data — full-sample factor IC, per-fold permutation importance, and leave-one-fold-out — all locate the same, single factor-and-window combination as whatever thin signal exists there. Corrected, both pieces of that signal are gone: value’s IC falls to -0.0051 ($t = -0.48$), so no factor has $|t| > 1$, and fold 7’s edge falls to a summed net monthly return of $+0.001$ (Corrections).

With that little standalone cross-sectional signal in any input, there is little for a non-linear model to combine into something a simple average does not already capture, independent of how correlated the inputs are with each other. Beyond that, monthly rebalancing over 15 years yields a modest number of independent observations (Section 5.4), and 8bps of turnover-based costs are a real, not token, headwind for the higher-turnover ML configurations (in the published run, 37% average monthly turnover for the baseline vs. over 100% for the best ML configuration) even though costs are not, per Section 5.1, the proximate cause of the non-result. None of this makes machine learning uninteresting for this problem in general — it is a comment on what three hand-picked factors, none individually distinguishable from zero in this particular sample, and 15 years of monthly data can support, not a general claim about machine learning in asset pricing.

Limitations.

1. **Coverage gap (Section 2).** 30.7% of point-in-time universe members are excluded for missing price history, with an unsigned bias direction for this dollar-neutral design. This is the most consequential limitation and is stated in the Abstract, not only here.
2. **Fundamentals coverage.** A further, separate 216-ticker SEC EDGAR gap leaves some names’ value/quality scores missing even where price data is available. This is an independent cause from the price gap in Section 2, not a consequence of it: Tiingo’s fallback is a *prices* API, so it was never going to close a *fundamentals* gap regardless of whether it had succeeded on prices.
3. **Ticker-symbol matching, not a permanent identifier.** All joins in this study are on ticker symbol. We checked two patterns under which this could contaminate a valid membership window with the wrong entity’s price data — non-contiguous membership runs, and previously flagged tickers whose fetched prices excluded their old window — and found none among those 20 names. *Corrected 2026-10-07*: the published version said we had checked “every structural pattern” and found none. COL shows otherwise: a single contiguous membership run, and a cached series that belongs to a different security from 2012-08 on (monthly-return correlation with Rockwell Collins of 0.069 over 75 overlapping months). Its effect on the results has not been isolated. Ticker matching cannot give the

guarantee that matching on CIK (for fundamentals) or a security-level identifier like CRSP PERMNO (for prices) would.

4. **The 0.95 deflated-Sharpe threshold was not itself pre-registered** (Section 4); we adopt it as a disclosed, conventional, post-hoc choice and show the conclusion is insensitive to it within any reasonable range.
5. **Small model grid, small factor set.** Nine configurations and three factors is a deliberately small, fully-enumerable search — appropriate for an honest deflated-Sharpe correction, but it also means the negative result says relatively little about what a much larger feature set or search might find.
6. **The published results contained a look-ahead leak, and were fragile in two further ways.** See Corrections: the leak in the value factor, the data-vintage sensitivity (0.241 to 0.096 for the published best configuration on re-downloaded data alone), and the unpinned scikit-learn version (0.147 under 1.9). The environment is now pinned; the data-vintage sensitivity is reported, not removed.

What would change the answer? A longer sample (more independent monthly observations directly lowers the minimum detectable Sharpe, per Section 5.4); a richer feature set beyond three factors, where non-linear interactions have more structure to exploit; resolving the coverage gap (Section 2) with full historical price vendor coverage, which could move the baseline and ML results in either direction given the unsigned bias direction; or testing in an asset class or sample with a real Sharpe 0.5+ edge to combine, since this design could not detect much less than that regardless of the method used to find it.

7. Conclusion

Machine learning does not beat a plain equal-weight linear combination of momentum, value, and quality in this pre-registered, walk-forward, cost-aware test. As published, the best of 8 pre-registered configurations showed a higher raw out-of-sample Sharpe than the linear baseline (0.241 against 0.066), but that advantage did not survive the deflated-Sharpe correction for the 9-configuration search, did not survive leaving out the single 12-month window responsible for the entire advantage, and was not backed by any stable relationship between the model and the three input factors. Corrected for a look-ahead leak in the value factor, the result is stronger: every configuration loses to the baseline outright (-0.590 to -0.361 , against -0.174), the baseline is itself negative, and the published edge has a documented cause. The pre-registered null survives both. A companion power analysis shows the study's sample size could not have detected a realistic edge in any case. The contribution here is not the specific null result, which is unsurprising given three well-known factors whose full-sample monthly rank ICs are each statistically indistinguishable from zero (Section 6) and a modest number of independent observations — it's a reproducible, pre-registered, cost-aware test of the question, with its own corrections (the in-draft one in Section 5.2, and the post-publication leak correction) and data limitations (Section 2, and the Abstract) disclosed rather than smoothed over.

References

- Bailey, D. H., and M. López de Prado. 2014. “The Deflated Sharpe Ratio: Correcting for Selection Bias, Backtest Overfitting, and Non-Normality.” *Journal of Portfolio Management* 40 (5): 94–107.
- Daniel, K., and T. J. Moskowitz. 2016. “Momentum Crashes.” *Journal of Financial Economics* 122 (2): 221–247.
- Fama, E. F., and K. R. French. 1993. “Common Risk Factors in the Returns on Stocks and Bonds.” *Journal of Financial Economics* 33 (1): 3–56.
- Gu, S., B. Kelly, and D. Xiu. 2020. “Empirical Asset Pricing via Machine Learning.” *Review of Financial Studies* 33 (5): 2223–2273.
- Jegadeesh, N., and S. Titman. 1993. “Returns to Buying Winners and Selling Losers: Implications for Stock Market Efficiency.” *Journal of Finance* 48 (1): 65–91.

Appendix: Reproducibility

- **Repository:** `github.com/robbasnet14/ml-vs-linear-factors` (public).
- **Pre-registration:** `PREREGISTRATION.md`, committed 2026-07-07 (commit `02647d5`), three weeks before any ML training (first ML commit `eac614c`, 2026-07-28). Deviations from the original text are logged, dated, in a `## Deviations from the original plan` section appended to that same file — nothing in the original text was edited.
- **Environment:** pinned in `backtester/requirements.txt` (Python 3.12.9, scikit-learn 1.5.1, pandas 2.3.2, numpy 2.0.1, scipy 1.15.2). The published results reproduce bit-for-bit only in this environment; under scikit-learn 1.9 the published best configuration’s Sharpe is 0.147 instead of 0.241.
- **Correction re-run:** `results/correction/` — inputs, all three modes’ return series, derived tables, and `FACTS.md` (every corrected number with its source). Re-run instructions are in `results/correction/README.md`.
- **Config:** `backtester/config.yaml` — universe SP500 2010-01-01 to 2024-12-31; 12-1 momentum, earnings-yield value, ROE quality; monthly rebalance, 10 deciles, dollar-neutral long/short; 8bps/trade; walk-forward 60-month train / 12-month test / 1-month embargo.
- **Tests:** 85 pytest tests, network-mocked, covering no-look-ahead, no-leakage (imputation fit on train folds only), survivorship-inclusive universe construction, the metrics themselves, and the factor IC computation in Section 6 (e.g. a perfectly rank-ordered synthetic month returns IC = 1.0, a reversed one returns -1.0).
- **Full results, all scripts, and every supplementary check** (coverage gap, ticker-identity check, Tiingo spot check, gross-vs-net, statistical power, leave-one-out, per-fold permutation importance, full-sample factor IC, cost sensitivity) are under `results/` and `backtester/scripts/`, each with a `README.md` documenting exactly how it was produced and how to reproduce it — including `results/comparison/factor_ic.csv` (Section 6’s factor IC table).

To reproduce end to end:

```
cd backtester
pip install -r requirements.txt
python -m pytest -q # see test count below
PYTHONPATH=. python scripts/run_backtest.py --config config.yaml
PYTHONPATH=. python scripts/run_ml_experiment.py --config config.yaml
PYTHONPATH=. python scripts/compare_ml_vs_baseline.py
PYTHONPATH=. python scripts/gross_vs_net.py
PYTHONPATH=. python scripts/dsr_power_table.py
PYTHONPATH=. python scripts/feature_importance.py
PYTHONPATH=. python scripts/subperiod_table.py
PYTHONPATH=. python scripts/cost_sensitivity.py
PYTHONPATH=. python scripts/factor_ic.py

# The correction (each mode about 20 minutes, offline, from the local data cache)
PYTHONPATH=. python scripts/correction_rerun.py --mode published # must
reproduce exactly
PYTHONPATH=. python scripts/correction_rerun.py --mode oldformula
PYTHONPATH=. python scripts/correction_rerun.py --mode corrected
PYTHONPATH=. python scripts/correction_tables.py
```